

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent

12411736

Kind Code

B2

Date of Patent

September 09, 2025

Inventor(s)

Dhuse; Greg R. et al.

Data reconstruction in a storage network and methods for use therewith

Abstract

A processor in a storage network operates by: receiving an access request for a data segment, wherein the data segment is encoded utilizing an error correcting information dispersal algorithm as a set of encoded data slices that are stored in a plurality of storage units of the storage network and wherein each encoded data slice of the set of encoded data slices includes a corresponding checksum of a plurality of checksums; retrieving, from the storage network, a subset of encoded data slices that includes a threshold number of encoded data slices of the set of encoded data slices; determining, based on ones of the plurality of checksums corresponding to the subset of encoded data slices, when the subset of encoded data slices includes at least one corrupted encoded data slice; retrieving from at least one of the plurality of storage units an addition number of encoded data slices required to generate a reconstructed data segment based on the subset of encoded data slices; generating the reconstructed data segment in accordance with the error correcting information dispersal algorithm, using the additional number of encoded data slices and at least some of the subset of encoded data slices; providing the reconstructed data segment in response to the access request; forming a reconstructed set of encoded data slices utilizing the error correcting information dispersal algorithm on the reconstructed data segment; and replacing the at least one corrupted encoded data slice with at least one reconstructed encoded data slice of the reconstructed set of encoded data slices.

Inventors: Dhuse; Greg R. (Chicago, IL), Thornton; Vance T. (Columbus, OH), Resch; Jason K. (Warwick, RI), Volvovski; Ilya (Chicago, IL), Hendrickson; Dustin M. (Chicago, IL), Quigley; John (Chicago, IL)

Applicant: Pure Storage, Inc. (Santa Clara, CA)

Family ID: 81079040

Assignee: Pure Storage, Inc. (Santa Clara, CA)

Appl. No.: 18/791544

Filed: August 01, 2024

Prior Publication Data

Document Identifier

US 20240394146 A1

Publication Date

Nov. 28, 2024

Related U.S. Application Data

continuation parent-doc US 17645563 20211222 US 12061519 child-doc US 18791544
continuation parent-doc US 16390530 20190422 US 11194662 20211207 child-doc US 16988135
continuation parent-doc US 14447890 20140731 US 10360180 20190723 child-doc US 16390530
continuation parent-doc US 13154725 20110607 US 10289688 20190514 child-doc US 14447890
continuation parent-doc US 12749592 20100330 US 8938591 20150120 child-doc US 14447890
continuation-in-part parent-doc US 16988135 20200807 US 11416339 20220816 child-doc US 17645563
continuation-in-part parent-doc US 16149667 20181002 ABANDONED child-doc US 17645563
continuation-in-part parent-doc US 15819810 20171121 US 11327674 20220510 child-doc US 16149667
continuation-in-part parent-doc US 13869655 20130424 US 10178083 20190108 child-doc US 15819810
continuation-in-part parent-doc US 12218594 20080716 US 7962641 20110614 child-doc US 13154725
continuation-in-part parent-doc US 12218200 20080714 US 8209363 20120626 child-doc US 12218594
continuation-in-part parent-doc US 12080042 20080331 US 8880799 20141104 child-doc US 12218594
continuation-in-part parent-doc US 11973613 20071009 US 8285878 20121009 child-doc US 12218594
continuation-in-part parent-doc US 11973621 20071009 US 7904475 20110308 child-doc US 12218594
continuation-in-part parent-doc US 11973622 20071009 US 8171101 20120501 child-doc US 12218594
continuation-in-part parent-doc US 11973542 20071009 US 9996413 20180612 child-doc US 12218594
continuation-in-part parent-doc US 11403684 20060413 US 7574570 20090811 child-doc US 12218594
continuation-in-part parent-doc US 11403391 20060413 US 7546427 20090609 child-doc US 12218594
continuation-in-part parent-doc US 11404071 20060413 US 7574579 20090811 child-doc US 12218594
continuation-in-part parent-doc US 11241555 20050930 US 7953937 20110531 child-doc US 12218594
us-provisional-application US 61655736 20120605
us-provisional-application US 61357430 20100622
us-provisional-application US 61237624 20090827

Publication Classification

Int. Cl.: **G06F11/10** (20060101); **G06F11/07** (20060101); **G06F11/14** (20060101); **G06F11/16** (20060101); **G06F16/13** (20190101); **H04L9/32** (20060101); **H04L9/40** (20220101); **H04L67/06** (20220101); **H04W12/041** (20210101); **H04W12/0431** (20210101); **H04W12/30** (20210101); G06F21/31 (20130101); G06F21/62 (20130101); H04L67/1097 (20220101); H04W12/10 (20210101)

U.S. Cl.:

CPC **G06F11/1092** (20130101); **G06F11/0727** (20130101); **G06F11/141** (20130101); **G06F11/167** (20130101); **G06F16/13** (20190101); **H04L9/3242** (20130101); **H04L9/3247** (20130101); **H04L9/3263** (20130101); **H04L9/3271** (20130101); **H04L63/06** (20130101); **H04L63/12** (20130101); **H04L67/06** (20130101); **H04W12/041** (20210101); **H04W12/0431** (20210101); **H04W12/35** (20210101); G06F16/137 (20190101); G06F21/31 (20130101); G06F21/6209 (20130101); G06F2211/1028 (20130101); H04L63/0428 (20130101); H04L67/1097 (20130101); H04L2209/043 (20130101); H04L2209/30 (20130101); H04L2209/34 (20130101); H04L2209/56 (20130101); H04L2209/80 (20130101); H04W12/10 (20130101)

Field of Classification Search

USPC: None

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
4092732	12/1977	Ouchi	N/A	N/A
5454101	12/1994	Mackay	N/A	N/A
5485474	12/1995	Rabin	N/A	N/A
5584008	12/1995	Shimada	N/A	N/A
5774643	12/1997	Lubbers	N/A	N/A
5802364	12/1997	Senator	N/A	N/A
5809285	12/1997	Hilland	N/A	N/A
5890156	12/1998	Rekieta	N/A	N/A
5987622	12/1998	Lo Verso	N/A	N/A
5991414	12/1998	Garay	N/A	N/A
6012159	12/1999	Fischer	N/A	N/A
6058454	12/1999	Gerlach	N/A	N/A
6128277	12/1999	Bruck	N/A	N/A
6175571	12/2000	Haddock	N/A	N/A
6192472	12/2000	Garay	N/A	N/A
6256688	12/2000	Suetaka	N/A	N/A
6272658	12/2000	Steele	N/A	N/A
6301604	12/2000	Nojima	N/A	N/A
6356949	12/2001	Katsandres	N/A	N/A
6366995	12/2001	Vilkov	N/A	N/A
6374336	12/2001	Peters	N/A	N/A
6415373	12/2001	Peters	N/A	N/A
6418539	12/2001	Walker	N/A	N/A

6449688	12/2001	Peters	N/A	N/A
6567948	12/2002	Steele	N/A	N/A
6571282	12/2002	Bowman-Amuah	N/A	N/A
6609223	12/2002	Wolfgang	N/A	N/A
6718361	12/2003	Basani	N/A	N/A
6760808	12/2003	Peters	N/A	N/A
6785768	12/2003	Peters	N/A	N/A
6785783	12/2003	Buckland	N/A	N/A
6826711	12/2003	Moulton	N/A	N/A
6836432	12/2003	Parker	N/A	N/A
6879596	12/2004	Dooply	N/A	N/A
6898667	12/2004	Umberger	N/A	N/A
6978366	12/2004	Ignatchenko	N/A	N/A
7000143	12/2005	Moulton	N/A	N/A
7003688	12/2005	Pittelkow	N/A	N/A
7024451	12/2005	Jorgenson	N/A	N/A
7024609	12/2005	Wolfgang	N/A	N/A
7080101	12/2005	Watson	N/A	N/A
7103824	12/2005	Halford	N/A	N/A
7103915	12/2005	Redlich	N/A	N/A
7111115	12/2005	Peters	N/A	N/A
7140044	12/2005	Redlich	N/A	N/A
7146644	12/2005	Redlich	N/A	N/A
7171493	12/2006	Shu	N/A	N/A
7222133	12/2006	Raipurkar	N/A	N/A
7225263	12/2006	Clymer	N/A	N/A
7240236	12/2006	Cutts	N/A	N/A
7272613	12/2006	Sim	N/A	N/A
7418649	12/2007	Li	N/A	N/A
7457835	12/2007	Toebe	N/A	N/A
7529834	12/2008	Birrell	709/227	G06F 11/1464
7533133	12/2008	Lanzatella	N/A	N/A
7574570	12/2008	Gladwin et al.	N/A	N/A
7581156	12/2008	Manasse	N/A	N/A
7607063	12/2008	Kikuchi	N/A	N/A
7636724	12/2008	De La Torre	N/A	N/A
7680822	12/2009	Vyas	N/A	N/A
7681104	12/2009	Sim-Tang	714/763	H04L 1/0057
7681105	12/2009	Sim-Tang	714/763	H03M 13/2909
7743275	12/2009	Tormasov	N/A	N/A
7831793	12/2009	Chakravarty	N/A	N/A
7865673	12/2010	Moore	N/A	N/A
7904475	12/2010	Gladwin	N/A	N/A
7925666	12/2010	Johnson	N/A	N/A
7945639	12/2010	Gavrilov	N/A	N/A
7962641	12/2010	Dhuse	N/A	N/A
8051362	12/2010	Li	N/A	N/A
8145818	12/2011	Murayama	N/A	N/A
8171101	12/2011	Gladwin	N/A	N/A
8209363	12/2011	Palthe	N/A	N/A

8214590	12/2011	Ulrich	N/A	N/A
8281181	12/2011	Resch	N/A	N/A
8281404	12/2011	Frey	N/A	N/A
8285878	12/2011	Gladwin	N/A	N/A
8335904	12/2011	Kitchen	N/A	N/A
8386840	12/2012	Stougie	N/A	N/A
8406421	12/2012	Kaymen	N/A	N/A
8429514	12/2012	Goel	N/A	N/A
8433849	12/2012	De Schrijver	N/A	N/A
8464133	12/2012	Grube	N/A	N/A
8620879	12/2012	Cairns	N/A	N/A
8694467	12/2013	Sun	N/A	N/A
8713405	12/2013	Healey	N/A	N/A
8856530	12/2013	Patti	N/A	N/A
8862837	12/2013	Marshak	N/A	N/A
8868508	12/2013	Drobychev	N/A	N/A
8880799	12/2013	Foster	N/A	N/A
8914632	12/2013	Shankar	N/A	N/A
8918478	12/2013	Ozzie	N/A	N/A
8935493	12/2014	Dolan	N/A	N/A
8938591	12/2014	Mark	711/155	G06F 3/0619
8972694	12/2014	Dolan	N/A	N/A
9098519	12/2014	Pavlov	N/A	G06F 3/0619
9235350	12/2015	Mark	N/A	N/A
9305069	12/2015	Zunger	N/A	G06F 16/27
9332422	12/2015	Bai	N/A	H04W 8/24
9372809	12/2015	Testardi	N/A	G06F 3/065
9792295	12/2016	Rus	N/A	N/A
9811262	12/2016	Rus	N/A	N/A
9996413	12/2017	Dhuse	N/A	G06F 11/1076
2002/0062422	12/2001	Butterworth	N/A	N/A
2002/0166079	12/2001	Ulrich	N/A	N/A
2003/0018927	12/2002	Gadir	N/A	N/A
2003/0037261	12/2002	Meffert	N/A	N/A
2003/0065617	12/2002	Watkins	N/A	N/A
2003/0065656	12/2002	de la Torre	N/A	N/A
2003/0084020	12/2002	Shu	N/A	N/A
2004/0024963	12/2003	Talagala	N/A	N/A
2004/0122917	12/2003	Menon	N/A	N/A
2004/0215998	12/2003	Buxton	N/A	N/A
2004/0228493	12/2003	Ma	N/A	N/A
2005/0055603	12/2004	Soran	N/A	N/A
2005/0100022	12/2004	Ramprashad	N/A	N/A
2005/0114594	12/2004	Corbett	N/A	N/A
2005/0125593	12/2004	Karpoff	N/A	N/A
2005/0131993	12/2004	Fatula, Jr.	N/A	N/A
2005/0132070	12/2004	Redlich	N/A	N/A
2005/0144382	12/2004	Schmisser	N/A	N/A
2005/0160329	12/2004	Briggs	N/A	N/A
2005/0210270	12/2004	Rohatgi	N/A	N/A

2005/0229069	12/2004	Hassner	N/A	N/A
2006/0041719	12/2005	Chui	N/A	N/A
2006/0047907	12/2005	Shiga	N/A	N/A
2006/0136448	12/2005	Cialini	N/A	N/A
2006/0156059	12/2005	Kitamura	N/A	N/A
2006/0224603	12/2005	Correll, Jr.	N/A	N/A
2007/0030734	12/2006	Sinclair	N/A	N/A
2007/0079081	12/2006	Gladwin	N/A	N/A
2007/0079082	12/2006	Gladwin	N/A	N/A
2007/0079083	12/2006	Gladwin	N/A	N/A
2007/0088970	12/2006	Buxton	N/A	N/A
2007/0113032	12/2006	Kameyama	N/A	N/A
2007/0174192	12/2006	Gladwin	N/A	N/A
2007/0214285	12/2006	Au	N/A	N/A
2007/0234110	12/2006	Soran	N/A	N/A
2007/0283167	12/2006	Venters, III	N/A	N/A
2008/0183975	12/2007	Foster	711/E12.001	G06F 11/08
2008/0235234	12/2007	Beedubail	N/A	N/A
2009/0037500	12/2008	Kirshenbaum	N/A	N/A
2009/0094250	12/2008	Dhuse	N/A	G06F 11/1076
2009/0094251	12/2008	Gladwin	N/A	N/A
2009/0094318	12/2008	Gladwin	N/A	N/A
2010/0023524	12/2009	Gladwin	711/E12.001	G06F 3/0659
2010/0088464	12/2009	Yang	N/A	N/A
2010/0138604	12/2009	Noguchi	N/A	N/A
2010/0218037	12/2009	Swartz	N/A	N/A
2010/0268692	12/2009	Resch	N/A	N/A
2010/0299313	12/2009	Orsini	N/A	N/A
2011/0029809	12/2010	Dhuse	714/6.1	H04L 67/1097
2011/0029840	12/2010	Ozzie	N/A	N/A
2011/0087948	12/2010	Murakami	N/A	N/A
2011/0126060	12/2010	Grube	N/A	N/A
2011/0225202	12/2010	Man	N/A	N/A
2011/0289122	12/2010	Grube	N/A	N/A
2012/0060072	12/2011	Simitci	N/A	N/A
2012/0131683	12/2011	Nassar	N/A	N/A
2013/0246470	12/2012	Price	N/A	N/A
2013/0275844	12/2012	Thornton	707/652	G06F 3/064
2015/0074216	12/2014	Park	709/208	H04L 41/5025
2015/0355979	12/2014	Volvovski	707/675	H04L 9/0897
2022/0114053	12/2021	Dhuse	N/A	G06F 16/1873

OTHER PUBLICATIONS

Chung; An Automatic Data Segmentation Method for 3D Measured Data Points; National Taiwan University; pp. 1-8; 1998. cited by applicant

Harrison; Lightweight Directory Access Protocol (LDAP): Authentication Methods and Security Mechanisms; IETF Network Working Group; RFC 4513; Jun. 2006; pp. 1-32. cited by applicant

Kubiatowicz, et al.; OceanStore: An Architecture for Global-Scale Persistent Storage; Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2000); Nov. 2000; pp. 1-12. cited by applicant

Legg; Lightweight Directory Access Protocol (LDAP): Syntaxes and Matching Rules; IETF Network Working Group; RFC 4517; Jun. 2006; pp. 1-50. cited by applicant

Plank, T1: Erasure Codes for Storage Applications; FAST2005, 4th Usenix Conference on File Storage Technologies; Dec. 13-16, 2005; pp. 1-74. cited by applicant

Rabin; Efficient Dispersal of Information for Security, Load Balancing, and Fault Tolerance; Journal of the Association for Computer Machinery; vol. 36, No. 2; Apr. 1989; pp. 335-348. cited by applicant

Satran, et al.; Internet Small Computer Systems Interface (iSCSI); IETF Network Working Group; RFC 3720; Apr. 2004; pp. 1-257. cited by applicant

Sciberras; Lightweight Directory Access Protocol (LDAP): Schema for User Applications; IETF Network Working Group; RFC 4519; Jun. 2006; pp. 1-33. cited by applicant

Sermersheim; Lightweight Directory Access Protocol (LDAP): The Protocol; IETF Network Working Group; RFC 4511; Jun. 2006; pp. 1-68. cited by applicant

Shamir; How to Share a Secret; Communications of the ACM; vol. 22, No. 11; Nov. 1979; pp. 612-613. cited by applicant

Smith; Lightweight Directory Access Protocol (LDAP): Uniform Resource Locator; IETF Network Working Group; RFC 4516; Jun. 2006; pp. 1-15. cited by applicant

Smith; Lightweight Directory Access Protocol (LDAP): String Representation of Search Filters; IETF Network Working Group; RFC 4515; Jun. 2006; pp. 1-12. cited by applicant

Wildi; Java iSCSI Initiator; Master Thesis; Department of Computer and Information Science, University of Konstanz; Feb. 2007; 60 pgs. cited by applicant

Xin, et al.; Evaluation of Distributed Recovery in Large-Scale Storage Systems; 13th IEEE International Symposium on High Performance Distributed Computing; Jun. 2004; pp. 172-181. cited by applicant

Zeilenga; Lightweight Directory Access Protocol (LDAP): Directory Information Models; IETF Network Working Group; RFC 4512; Jun. 2006; pp. 1-49. cited by applicant

Zeilenga; Lightweight Directory Access Protocol (LDAP): Internationalized String Preparation; IETF Network Working Group; RFC 4518; Jun. 2006; pp. 1-14. cited by applicant

Zeilenga; Lightweight Directory Access Protocol (LDAP): String Representation of Distinguished Names; IETF Network Working Group; RFC 4514; Jun. 2006; pp. 1-15. cited by applicant

Zeilenga; Lightweight Directory Access Protocol (LDAP): Technical Specification Road Map; IETF Network Working Group; RFC 4510; Jun. 2006; pp. 1-8. cited by applicant

Primary Examiner: Harper; Eliyah S.

Attorney, Agent or Firm: Katz Ruby & Carle LLP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) The present U.S. Utility Patent application claims priority pursuant to 35 U.S.C. § 120 as a continuation of U.S. Utility application Ser. No. 17/645,563, entitled “Reconstructing Data Segments in a Storage Network and Methods for use Therewith”, filed Dec. 22, 2021, which is a continuation-in-part of U.S. Utility application Ser. No. 16/149,667, entitled “Utilizing Concentric Storage Pools in a Dispersed Storage Network,”, filed Oct. 2, 2018, which is a continuation-in-part of U.S. Utility application Ser. No. 15/819,810, entitled “Storage Vault Tiering and Data Migration in a Distributed Storage Network,” filed Nov. 21, 2017, issued as U.S. Pat. No. 11,327,674 on May 10, 2022, which is a continuation-in-part of U.S. Utility application Ser. No. 13/869,655, entitled “Updating Access Control

Information within a Dispersed Storage Unit,” filed Apr. 24, 2013, issued as U.S. Pat. No. 10,178,083 on Jan. 8, 2019, which claims priority pursuant to 35 U.S.C. § 119 (c) to U.S. Provisional Application No. 61/655,736, entitled “Storing Data in a Layered Distributed Storage and Task Network”, filed Jun. 5, 2012, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility Patent Application for all purposes. (2) U.S. Utility application Ser. No. 17/645,563 also claims priority pursuant to 35 U.S.C. § 120 as a continuation-in-part of U.S. Utility application Ser. No. 16/988,135, entitled “Validating Requests Based on Stored Vault Information”, filed Aug. 7, 2020, issued as U.S. Pat. No. 11,416,339 on Aug. 16, 2022, which is a continuation of U.S. Utility application Ser. No. 16/390,530, entitled “Digest Listing Decomposition”, filed Apr. 22, 2019, issued as U.S. Pat. No. 11,194,662 on Dec. 7, 2021, which is a continuation of U.S. Utility application Ser. No. 14/447,890, entitled “Digest Listing Decomposition”, filed Jul. 31, 2014, issued as U.S. Pat. No. 10,360,180 on Jul. 23, 2019, which is a continuation of U.S. Utility application Ser. No. 13/154,725, entitled, “Metadata Access In A Dispersed Storage Network”, filed Jun. 7, 2011, issued as U.S. Pat. No. 10,289,688 on May 14, 2019, which claims priority pursuant to 35 U.S.C. § 119 (c) to U.S. Provisional Application No. 61/357,430, entitled “Dispersal Method In A Dispersed Storage System”, filed Jun. 22, 2010, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility Patent Application for all purposes. (3) U.S. Utility application Ser. No. 14/447,890 also claims priority pursuant to 35 U.S.C. § 120 as a continuation-in-part of U.S. Utility application Ser. No. 12/749,592, entitled “Dispersed Storage Processing Unit And Methods With Data Aggregation For Use In A Dispersed Storage System”, filed Mar. 30, 2010, issued as U.S. Pat. No. 8,938,591 on Jan. 20, 2015, which claims priority pursuant to 35 U.S.C. § 119 (e) to U.S. Provisional Application No. 61/237,624, entitled “Dispersed Storage Unit And Methods With Metadata Separation For Use In A Dispersed Storage System”, filed Aug. 27, 2009, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility Patent Application for all purposes. (4) U.S. Utility application Ser. No. 12/749,592 also claims priority pursuant to 35 U.S.C. § 120 as a continuation-in-part of U.S. Utility application Ser. No. 12/218,594, entitled “Streaming Media Software Interface To A Dispersed Data Storage Network”, filed Jul. 16, 2008, issued as U.S. Pat. No. 7,962,641 on Jun. 14, 2011, which claims priority pursuant to 35 U.S.C. § 120 as a continuation-in-part of: 1. U.S. Utility application Ser. No. 11/973,613, entitled “Block Based Access To A Dispersed Data Storage Network”, filed Oct. 9, 2007, issued as U.S. Pat. No. 8,285,878 on Oct. 9, 2012; 2. U.S. Utility application Ser. No. 11/973,622, entitled “Smart Access To A Dispersed Data Storage Network”, filed Oct. 9, 2007, issued as U.S. Pat. No. 8,171,101 on May 1, 2012; 3. U.S. Utility application Ser. No. 11/973,542, entitled “Ensuring Data Integrity On A Dispersed Storage Network”, filed Oct. 9, 2007, issued as U.S. Pat. No. 9,996,413 on Jun. 12, 2018; 4. U.S. Utility application Ser. No. 11/973,621, entitled “Virtualized Storage Vaults On A Dispersed Data Storage Network”, filed Oct. 9, 2007, issued as U.S. Pat. No. 7,904,475 on Mar. 8, 2011; 5. U.S. Utility application Ser. No. 11/241,555, entitled “System, Methods, And Apparatus For Subdividing Data For Storage In A Dispersed Data Storage Grid”, filed Sep. 30, 2005, issued as U.S. Pat. No. 7,953,937 on May 31, 2011; 6. U.S. Utility application Ser. No. 11/403,684, entitled “Billing System For Information Dispersal System”, filed Apr. 13, 2006, issued as U.S. Pat. No. 7,574,570 on Aug. 11, 2009; 7. U.S. Utility application Ser. No. 11/404,071, entitled “Metadata Management System For An Information Dispersed Storage System”, filed Apr. 13, 2006, issued as U.S. Pat. No. 7,574,579 on Aug. 11, 2009; 8. U.S. Utility application Ser. No. 11/403,391, entitled “System For Rebuilding Dispersed Data”, filed Apr. 13, 2006, issued as U.S. Pat. No. 7,546,427 on Jun. 9, 2009; 9. U.S. Utility application Ser. No. 12/080,042, entitled “Rebuilding Data On A Dispersed Storage Network”, filed Mar. 31, 2008, issued as U.S. Pat. No. 8,880,799 on Nov. 4, 2014, and 10. U.S. Utility application Ser. No. 12/218,200, entitled “File System Adapted For Use With A Dispersed Data Storage Network”, filed Jul. 14, 2008, issued as U.S. Pat. No. 8,209,363 on Jun. 26, 2012; (5) All of the above are hereby

incorporated herein by reference in their entirety and made part of the present U.S. Utility Patent Application for all purposes. (6) In addition, U.S. Utility application Ser. No. 16/988,135 is related to the following U.S. patent applications that are commonly owned: 1. “Dispersed Storage Unit And Methods With Metadata Separation For Use In A Dispersed Storage System”, application Ser. No. 12/749,583, filed on Mar. 30, 2010, issued as U.S. Pat. No. 9,235,350 on Jan. 12, 2016. 2. “Dispersed Storage Processing Unit And Methods With Operating System Diversity For Use In A Dispersed Storage System”, application Ser. No. 12/749,606, filed on Mar. 30, 2010, issued as U.S. Pat. No. 9,690,513 on Jun. 27, 2017. 3. “Dispersed Storage Processing Unit And Methods With Geographical Diversity For Use In A Dispersed Storage System”, application Ser. No. 12/749,625, and filed on Mar. 30, 2010, issued as U.S. Pat. No. 9,772,791 on Sep. 26, 2017. (7) All of the above are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility Patent Application for all purposes.

FIELD OF THE DISCLOSURE

(1) The present disclosure relates generally to systems, apparatus, and methods for distributed data storage, and more particularly to systems, apparatus, and methods for distributed data storage using an information dispersal algorithm so that no one location will store an entire copy of stored data, and more particularly still to systems, apparatus, and methods for ensuring data integrity on a dispersed data storage network.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- (1) Although the characteristic features of this disclosure will be particularly pointed out in the claims, and the disclosure itself—the manner in which it may be made and used, may be better understood by referring to the following description taken in connection with the accompanying drawings forming a part hereof, wherein like reference numerals refer to like parts throughout the several views and in which:
- (2) FIG. 1 is a network diagram of a dispersed data storage network constructed in accordance with an example of the disclosed disclosure;
- (3) FIG. 2 illustrates the use of checksums on a data segment as well as on an arbitrary number of data slices created from the data segment;
- (4) FIG. 3 is a flowchart illustrating the process by which a corrupted data segment can be rebuilt by a dispersed data storage network constructed in accordance with an example of the disclosed disclosure;
- (5) FIGS. 4A-4C collectively illustrates a read operation from a dispersed data storage network constructed in accordance with an example of the disclosed disclosure;
- (6) FIGS. 5A-5B collectively illustrates a write operation from a dispersed data storage network constructed in accordance with an example of the disclosed disclosure;
- (7) FIGS. 6A-6B collectively illustrates an alternative process by which corrupted data slices may be recreated.

DETAILED DESCRIPTION OF THE ILLUSTRATED EXAMPLES

(8) Storing data in digital form is a well-known problem associated with all computer systems, and numerous solutions to this problem are known in the art. The simplest solution involves merely storing digital data in a single location, such as a punch film, hard drive, or FLASH memory device. However, storage of data in a single location is inherently unreliable. The device storing the data can malfunction or be destroyed through natural disasters, such as a flood, or through a malicious act, such as arson. In addition, digital data is generally stored in a usable file, such as a document that can be opened with the appropriate word processing software, or a financial ledger that can be opened with the appropriate spreadsheet software. Storing an entire usable file in a

single location is also inherently insecure as a malicious hacker only need compromise that one location to obtain access to the usable file.

(9) To address reliability concerns, digital data is often “backed-up,” i.e., an additional copy of the digital data is made and maintained in a separate physical location. For example, a backup tape of all network drives may be made by a small office and maintained at the home of a trusted employee. When a backup of digital data exists, the destruction of either the original device holding the digital data or the backup will not compromise the digital data. However, the existence of the backup exacerbates the security problem, as a malicious hacker can choose between two locations from which to obtain the digital data. Further, the site where the backup is stored may be far less secure than the original location of the digital data, such as in the case when an employee stores the tape in her home.

(10) Another method used to address reliability and performance concerns is the use of a Redundant Array of Independent Drives (“RAID”). RAID refers to a collection of data storage schemes that divide and replicate data among multiple storage units. Different configurations of RAID provide increased performance, improved reliability, or both increased performance and improved reliability. In certain configurations of RAID, when digital data is stored, it is split into multiple units, referred to as “stripes,” each of which is stored on a separate drive. Data striping is performed in an algorithmically certain way so that the data can be reconstructed. While certain RAID configurations can improve reliability, RAID does nothing to address security concerns associated with digital data storage.

(11) Encrypted data is mathematically coded so that only users with access to a certain key can decrypt and use the data. Common forms of encryption include DES, AES, RSA, and others. While modern encryption methods are difficult to break, numerous instances of successful attacks are known, some of which have resulted in valuable data being compromised.

(12) Digitally stored data is subject to degradation over time, although such degradation tends to be extremely minor and the time periods involved tend to be much longer than for analog data storage. Nonetheless, if a single bit within a file comprised of millions of bits changes from a zero to a one or vice versa, the integrity of the file has been compromised, and its usability becomes suspect. Further, errors occur more frequently when digital data is transmitted due to noise in the transmission medium. Various prior art techniques have been devised to detect when a digital data segment has been compromised. One early form of error detection is known as parity, wherein a single bit is appended to each transmitted byte or word of data. The parity bit is set so that the total number of one bits in the transmitted byte or word is either even or odd. The receiving processor then checks the received byte or word for the appropriate parity, and, if it is incorrect, asks that the byte or word be resent.

(13) Another form of error detection is the use of a checksum. There are many different types of checksums including classic checksums, cryptographic hash functions, digital signatures, cyclic redundancy checks, and the use of human readable “check digits” by the postal service and libraries. All of these techniques involve performing a mathematical calculation over an entire data segment to arrive at a checksum, which is appended to the data segment. For stored data, the checksum for the data segment can be recalculated periodically, and checked against the previously calculated checksum appended to the data segment. For transmitted data, the checksum is calculated by the transmitter and appended to the data segment. The receiver then recalculates the checksum for the received data segment, and if it does not match the checksum appended to the data segment, requests that it be retransmitted.

(14) In 1979, two researchers independently developed a method for splitting data among multiple recipients called “secret sharing.” One of the characteristics of secret sharing is that a piece of data may be split among n recipients, but cannot be known unless at least t recipients share their data, where $n > t$. For example, a trivial form of secret sharing can be implemented by assigning a single random byte to every recipient but one, who would receive the actual data byte after it had

been bitwise exclusive orred with the random bytes. In other words, for a group of four recipients, three of the recipients would be given random bytes, and the fourth would be given a byte calculated by the following formula:

$$(15) s' = s \oplus r_a \oplus r_b \oplus r_c,$$

where s is the original source data, $r_{sub.a}$, $r_{sub.b}$, and $r_{sub.c}$ are random bytes given to three of the four recipients, and s' is the encoded byte given to the fourth recipient. The original byte s can be recovered by bitwise exclusive-oring all four bytes together.

(16) The problem of reconstructing data stored on a digital medium that is subject to damage has also been addressed in the prior art. In particular, Reed-Solomon and Cauchy Reed-Solomon coding are two well-known methods of dividing encoded information into multiple slices so that the original information can be reassembled even if all of the slices are not available. Reed-Solomon coding, Cauchy Reed-Solomon coding, and other data coding techniques are described in "Erasure Codes for Storage Applications," by Dr. James S. Plank.

(17) While dispersed data storage networks (DDSN's) can theoretically be implemented to provide any desired level of reliability, practical considerations tend to make this impossible in prior art solutions. For example, DDSNs rely on storage media to store data slices. This storage media, like all storage media, will degrade over time. Furthermore, DDSN's rely on numerous transmissions to physically disparate slice servers, and data slices may become corrupted during transmissions. While TCP utilizes a CRC in every transmitted packet, the reliability provided by this CRC is not sufficient for critical data storage.

(18) The disclosed disclosure achieves its objectives by providing an improved method for insuring the integrity of data stored on a dispersed data storage network. A checksum is calculated for a data segment to be written to a DDSN. The checksum is appended to the data segment, which is sliced into a plurality of data slices. A second set of checksums is calculated for and appended to the different data slices, which are then transmitted to different slice servers. For each receiving slice server, a checksum is calculated for the received data slice, and compared to the checksum appended to the received data slice. If the checksums vary, the receiving slice server marks the data slice as corrupted, and requests that the corrupted data slice be resent.

(19) In another aspect of the disclosed disclosure, a distributed computer system implements a dispersed data storage network. In this system, a rebuilder application periodically recalculates checksums for data slices stored on a plurality of slice servers. Where the calculated checksum does not match the checksum appended to a stored data slice, the data slice is marked as corrupted. The rebuilder application then identifies the stored data segment associated with the corrupted data slice, and issues read requests to other slice servers holding data slices corresponding to the identified data segment. The data segment is rebuilt and re-sliced, and any slice servers containing corrupted data are sent new data slices to replace the corrupted data slices.

(20) Turning to the Figures, and to FIG. 1 in particular, a distributed computer system implementing a dispersed data storage grid **100** is shown. An arbitrary number of slice servers **150-162** store data slices sent to them by networked source computers **102, 104, 106**. As illustrated, some number of grid access computers **120, 122** allows access to the slice servers **150-162** by the source computers **102, 104, 106**.

(21) As explained herein, the disclosed disclosure works to ensure integrity of data stored in a DDSN not only by using checksums on each stored data segment as well as the constituent data slices, but also by reconstructing corrupted data slices as well. In accordance with the disclosed disclosure, grid access computers **120, 122** will calculate a checksum for each data segment to be stored, and append the checksum to the data segment prior to slicing. The data segment is then sliced in accordance with an information dispersal algorithm, and checksums are calculated and appended to each of the data slices. The data slices are then forwarded to slice servers **150-162**, where the data slices are stored.

(22) In addition, grid access computers **120, 122** also recreate data slices that have become

corrupted, or were destroyed. If during operation of the DDSN **100**, it is detected that a particular data slice has been corrupted or destroyed, a different data slice will be requested from a different slice server **150-162**. Assuming that sufficient non-corrupted data slices exist to successfully reconstruct the original data segment, the reconstructed data segment will be re-sliced, and the corrupted data slice will be replaced with a non-corrupted version. Further, a rebuilder application operating within the DDSN periodically walks through all data slices stored on the DDSN. When a corrupted data slice is found, the rebuilder application identifies the data segment corresponding to the corrupted data slice, rebuilds the identified data segment, and rewrites the corrupted slice.

(23) FIG. **2** depicts the use of a checksum **220** on a data segment **230**, as well as on the data slices **250-257** that the data segment **230** was divided into. Assuming that a data segment **230** is being written to a DDSN, a checksum **220** will be calculated for and appended to the data segment **230**, thereby forming a “certified data segment.” The certified data segment **230** will then be sliced as one piece of data resulting in data slices **250-257**, i.e., when the data slices **250-257** are recombined, both the data segment **230** and data segment checksum **220** will be recovered. A checksum **240-247** is then calculated for, and appended to each data slice **250-257**, forming “certified data slices” **260-267**. The certified data slices **260-267** will then be sent to different slice servers.

(24) FIG. **3** depicts one possible process by which corrupted slices may be recreated. During a read operation of the DDSN, a client requests a slice from a slice server in step **302**. In step **303**, the slice server transmits the requested slice to the client, which calculates a checksum for the requested data slice. In step **304**, the calculated checksum is compared to the checksum appended to the stored data slice, and if the two checksums match, the read proceeds as normal in step **306**. However, if the two checksums do not match, the slice server will transmit a message to a rebuilder application operating on the DDSN indicating that the requested data slice is corrupted in step **308**, and return “Failure” to the querying server. The corrupted slice may be rewritten asynchronously as discussed in the text addressing FIG. **6**. In step **310**, the querying server determines if an alternative slice can be read from a different slice server. If an alternative slice does not exist, the querying server will report an error in step **312**. In step **314**, the querying computer reads the alternative slice.

(25) FIGS. **4A-4C** show the process by which a DDSN, constructed in accordance with the disclosed disclosure and used in conjunction with the with the process depicted in FIG. **3**, could fulfill a read request. In step **402**, a read request is received. Contained within the read request will be information sufficient to determine which slice servers contain applicable data, as well as the minimum number of data slices that must be retrieved before the requested data can be reconstructed. Further information on one method that can be used to associate data requests with particular slice servers is contained in U.S. patent application Ser. No. 11/973,621, titled “VIRTUALIZED DATA STORAGE VAULTS ON A DISPERSED DATA STORAGE NETWORK,” filed on Oct. 9, 2007 and assigned to Pure Storage, Inc. In step **404**, the variable m is initialized to the minimum number of slices required to construct the requested data segment. As described herein, for each successfully received and validated slice, m is decremented.

(26) In step **406**, a list of slice servers each holding a required data slice that has yet to be received is assembled, and in step **408**, the list is ordered by any applicable criteria. Further information on criteria by which the list may be ordered is contained in U.S. patent application Ser. No. 11/973,622, titled “SMART ACCESS TO A DISPERSED DATA STORAGE NETWORK,” filed on Oct. 9, 2007 and assigned to Pure Storage, Inc. In step **410**, read requests are issued to the first k slice servers on the assembled list, where k is at least equal to m , the minimum number of data slices needed to reconstruct the requested data segment, but could be as large as n , the number of data slices that have data relevant to the requested data segment. In step **412**, r data slices are received, and in step **414** the number of received data slices r is subtracted from the variable m . In step **416**, m is compared to zero, and if m is greater than or equal to zero, execution returns to step

406 and proceeds as normal from there. However, if m is equal to zero, a collection of data transformations may optionally be applied to the received slices in step **418**. The applied data transformations can include decryption, decompression, and integrity checking. In accordance with the disclosed disclosure, each data slice includes a cyclical redundancy check ("CRC"), or other form of checksum appended to the data contained in the slice. This checksum will be compared against a checksum calculated by the receiving slice server against the received data to ensure that the data was not corrupted during the transmission process.

(27) In step **420**, it is determined if the applied data transformations were successful for all of the received data slices. If the applied data transformations were not successful for some of the received slices, m is incremented by this number in step **422**, and execution is resumed at step **406**. The data transformations could fail, for example, if an integrity check revealed that a received data slice was corrupted. However, if the applied data transformations were successful for all received data slices, the received slices are assembled into the requested block of data in step **424**. The same or different data transformations may optionally be applied to the assembled data block in step **426**, which completes the read process. In accordance with the disclosed disclosure, a checksum for the data segment will be calculated and compared to a checksum appended to the assembled data segment.

(28) In FIGS. 5A-5B the process by which a DDSN, constructed in accordance with the disclosed disclosure, could write data to a network of slice servers is illustrated. In step **502** a data segment write request is received. Included in this write request is information sufficient to determine which slice servers the data segment should be written to, as well as information required by the information dispersal algorithm to divide the data segment, i.e., the number of slices to be written, referred to as n , as well as the minimum number of slices that are required to recover the data, referred to as m . Further information on one method that can be used to associate data writes with particular slice servers is contained in U.S. patent application Ser. No. 11/973,621, titled "VIRTUALIZED DATA STORAGE VAULTS ON A DISPERSED DATA STORAGE NETWORK," filed on Oct. 9, 2007 and assigned to Pure Storage, Inc.

(29) A number of data transformations may optionally be applied to each block in step **506**, and an information dispersal algorithm is applied in step **508**. In particular, the Cauchy Reed-Solomon dispersal algorithm could be applied to the data segment, resulting in a predetermined number of data slices. In step **510**, a number of data transformations are optionally applied to each data slice.

(30) In the disclosed system, writes are performed transactionally, meaning that a minimum number of data slices t must be successfully written before a write is deemed complete. Normally, the number of data slices that must be successfully written will be set to n , i.e., the number of slices that the data segment was originally divided into. However, this number can be configured by the user to a lesser number, down to the minimum number of slices required to reconstruct the data. This would allow the user to continue using the DDSN during a minor network outage where one or more slice servers were unavailable. Slices that could not be immediately transmitted and stored could be queued and transmitted when the network outage cleared. In step **512**, a write transaction is initiated to the data storage grid. As discussed herein, all slice servers are simultaneously contacted, and in step **514**, a confirmation that at least t receiving slice servers are prepared to begin the write transaction, i.e., to store each slice, must be received, or the transaction is rolled back in step **516**.

(31) In step **520** data slices are transmitted to the slice servers that indicated their ability to receive and store slices. The number of slice servers that successfully received and stored their assigned data slices is checked in step **522**, and if less than t slices are successfully stored, the transaction is rolled back in step **516**. In step **524**, a commit transaction is begun on all servers with successful writes. If the commit transaction fails, an error is logged in step **528**. Otherwise, the write transaction was successful.

(32) FIGS. 6A-6B are a flow chart illustrating an alternative process by which corrupted data slices

may be recreated. In step **602**, a scan of data slices is initiated by a rebuilder application operating somewhere on the DDSN. If no corrupted data slice is found, the corrupted slice recreation process is exited in step **605**. However, if a corrupted slice is detected because of a failed integrity check, execution proceeds to step **606**, where a grid access computer determines what data segment corresponds to the corrupted data slice, and reads that data segment from the DDSN. The grid access computer then reconstructs the data segment in step **608**. In step **610**, the data segment is re-sliced, and the grid access computer rewrites a non-corrupted version of the corrupted data slice to the appropriate slice server in step **612**.

(33) The foregoing description of the disclosure has been presented for purposes of illustration and description, and is not intended to be exhaustive or to limit the disclosure to the precise form disclosed. The description was selected to best explain the principles of the disclosure and practical application of these principles to enable others skilled in the art to best utilize the disclosure in various examples and various modifications as are suited to the particular use contemplated. It is intended that the scope of the disclosure not be limited by the specification, but be defined by the claims set forth below.

(34) It is noted that terminologies as may be used herein such as bit stream, stream, signal sequence, etc. (or their equivalents) have been used interchangeably to describe digital information whose content corresponds to any of a number of desired types (e.g., data, video, speech, text, graphics, audio, etc. any of which may generally be referred to as ‘data’).

(35) As may be used herein, the terms “substantially” and “approximately” provides an industry-accepted tolerance for its corresponding term and/or relativity between items. For some industries, an industry-accepted tolerance is less than one percent and, for other industries, the industry-accepted tolerance is 10 percent or more. Other examples of industry-accepted tolerance range from less than one percent to fifty percent. Industry-accepted tolerances correspond to, but are not limited to, component values, integrated circuit process variations, temperature variations, rise and fall times, thermal noise, dimensions, signaling errors, dropped packets, temperatures, pressures, material compositions, and/or performance metrics. Within an industry, tolerance variances of accepted tolerances may be more or less than a percentage level (e.g., dimension tolerance of less than $\pm 1\%$). Some relativity between items may range from a difference of less than a percentage level to a few percent. Other relativity between items may range from a difference of a few percent to magnitude of differences.

(36) As may also be used herein, the term(s) “configured to”, “operably coupled to”, “coupled to”, and/or “coupling” includes direct coupling between items and/or indirect coupling between items via an intervening item (e.g., an item includes, but is not limited to, a component, an element, a circuit, and/or a module) where, for an example of indirect coupling, the intervening item does not modify the information of a signal but may adjust its current level, voltage level, and/or power level. As may further be used herein, inferred coupling (i.e., where one element is coupled to another element by inference) includes direct and indirect coupling between two items in the same manner as “coupled to”.

(37) As may even further be used herein, the term “configured to”, “operable to”, “coupled to”, or “operably coupled to” indicates that an item includes one or more of power connections, input(s), output(s), etc., to perform, when activated, one or more its corresponding functions and may further include inferred coupling to one or more other items. As may still further be used herein, the term “associated with”, includes direct and/or indirect coupling of separate items and/or one item being embedded within another item.

(38) As may be used herein, the term “compares favorably”, indicates that a comparison between two or more items, signals, etc., provides a desired relationship. For example, when the desired relationship is that signal **1** has a greater magnitude than signal **2**, a favorable comparison may be achieved when the magnitude of signal **1** is greater than that of signal **2** or when the magnitude of signal **2** is less than that of signal **1**. As may be used herein, the term “compares unfavorably”,

indicates that a comparison between two or more items, signals, etc., fails to provide the desired relationship.

(39) As may be used herein, one or more claims may include, in a specific form of this generic form, the phrase “at least one of a, b, and c” or of this generic form “at least one of a, b, or c”, with more or less elements than “a”, “b”, and “c”. In either phrasing, the phrases are to be interpreted identically. In particular, “at least one of a, b, and c” is equivalent to “at least one of a, b, or c” and shall mean a, b, and/or c. As an example, it means: “a” only, “b” only, “c” only, “a” and “b”, “a” and “c”, “b” and “c”, and/or “a”, “b”, and “c”.

(40) As may also be used herein, the terms “processing module”, “processing circuit”, “processor”, “processing circuitry”, and/or “processing unit” may be a single processing device or a plurality of processing devices. Such a processing device may be a microprocessor, micro-controller, digital signal processor, microcomputer, central processing unit, field programmable gate array, programmable logic device, state machine, logic circuitry, analog circuitry, digital circuitry, and/or any device that manipulates signals (analog and/or digital) based on hard coding of the circuitry and/or operational instructions. The processing module, module, processing circuit, processing circuitry, and/or processing unit may be, or further include, memory and/or an integrated memory element, which may be a single memory device, a plurality of memory devices, and/or embedded circuitry of another processing module, module, processing circuit, processing circuitry, and/or processing unit. Such a memory device may be a read-only memory, random access memory, volatile memory, non-volatile memory, static memory, dynamic memory, flash memory, cache memory, and/or any device that stores digital information. Note that if the processing module, module, processing circuit, processing circuitry, and/or processing unit includes more than one processing device, the processing devices may be centrally located (e.g., directly coupled together via a wired and/or wireless bus structure) or may be distributedly located (e.g., cloud computing via indirect coupling via a local area network and/or a wide area network). Further note that if the processing module, module, processing circuit, processing circuitry and/or processing unit implements one or more of its functions via a state machine, analog circuitry, digital circuitry, and/or logic circuitry, the memory and/or memory element storing the corresponding operational instructions may be embedded within, or external to, the circuitry comprising the state machine, analog circuitry, digital circuitry, and/or logic circuitry. Still further note that, the memory element may store, and the processing module, module, processing circuit, processing circuitry and/or processing unit executes, hard coded and/or operational instructions corresponding to at least some of the steps and/or functions illustrated in one or more of the Figures. Such a memory device or memory element can be included in an article of manufacture.

(41) One or more embodiments have been described above with the aid of method steps illustrating the performance of specified functions and relationships thereof. The boundaries and sequence of these functional building blocks and method steps have been arbitrarily defined herein for convenience of description. Alternate boundaries and sequences can be defined so long as the specified functions and relationships are appropriately performed. Any such alternate boundaries or sequences are thus within the scope and spirit of the claims. Further, the boundaries of these functional building blocks have been arbitrarily defined for convenience of description. Alternate boundaries could be defined as long as the certain significant functions are appropriately performed. Similarly, flow diagram blocks may also have been arbitrarily defined herein to illustrate certain significant functionality.

(42) To the extent used, the flow diagram block boundaries and sequence could have been defined otherwise and still perform the certain significant functionality. Such alternate definitions of both functional building blocks and flow diagram blocks and sequences are thus within the scope and spirit of the claims. One of average skill in the art will also recognize that the functional building blocks, and other illustrative blocks, modules and components herein, can be implemented as illustrated or by discrete components, application specific integrated circuits, processors executing

appropriate software and the like or any combination thereof.

(43) In addition, a flow diagram may include a “start” and/or “continue” indication. The “start” and “continue” indications reflect that the steps presented can optionally be incorporated in or otherwise used in conjunction with one or more other routines. In addition, a flow diagram may include an “end” and/or “continue” indication. The “end” and/or “continue” indications reflect that the steps presented can end as described and shown or optionally be incorporated in or otherwise used in conjunction with one or more other routines. In this context, “start” indicates the beginning of the first step presented and may be preceded by other activities not specifically shown. Further, the “continue” indication reflects that the steps presented may be performed multiple times and/or may be succeeded by other activities not specifically shown. Further, while a flow diagram indicates a particular ordering of steps, other orderings are likewise possible provided that the principles of causality are maintained.

(44) The one or more embodiments are used herein to illustrate one or more aspects, one or more features, one or more concepts, and/or one or more examples. A physical embodiment of an apparatus, an article of manufacture, a machine, and/or of a process may include one or more of the aspects, features, concepts, examples, etc. described with reference to one or more of the embodiments discussed herein. Further, from figure to figure, the embodiments may incorporate the same or similarly named functions, steps, modules, etc. that may use the same or different reference numbers and, as such, the functions, steps, modules, etc. may be the same or similar functions, steps, modules, etc. or different ones.

(45) Unless specifically stated to the contra, signals to, from, and/or between elements in a figure of any of the figures presented herein may be analog or digital, continuous time or discrete time, and single-ended or differential. For instance, if a signal path is shown as a single-ended path, it also represents a differential signal path. Similarly, if a signal path is shown as a differential path, it also represents a single-ended signal path. While one or more particular architectures are described herein, other architectures can likewise be implemented that use one or more data buses not expressly shown, direct connectivity between elements, and/or indirect coupling between other elements as recognized by one of average skill in the art.

(46) The term “module” is used in the description of one or more of the embodiments. A module implements one or more functions via a device such as a processor or other processing device or other hardware that may include or operate in association with a memory that stores operational instructions. A module may operate independently and/or in conjunction with software and/or firmware. As also used herein, a module may contain one or more sub-modules, each of which may be one or more modules.

(47) As may further be used herein, a computer readable memory includes one or more memory elements. A memory element may be a separate memory device, multiple memory devices, or a set of memory locations within a memory device. Such a memory device may be a read-only memory, random access memory, volatile memory, non-volatile memory, static memory, dynamic memory, flash memory, cache memory, a quantum register or other quantum memory and/or any other device that stores data in a non-transitory manner. Furthermore, the memory device may be in a form of a solid-state memory, a hard drive memory or other disk storage, cloud memory, thumb drive, server memory, computing device memory, and/or other non-transitory medium for storing data. The storage of data includes temporary storage (i.e., data is lost when power is removed from the memory element) and/or persistent storage (i.e., data is retained when power is removed from the memory element). As used herein, a transitory medium shall mean one or more of: (a) a wired or wireless medium for the transportation of data as a signal from one computing device to another computing device for temporary storage or persistent storage; (b) a wired or wireless medium for the transportation of data as a signal within a computing device from one element of the computing device to another element of the computing device for temporary storage or persistent storage; (c) a wired or wireless medium for the transportation of data as a signal from one computing device to

another computing device for processing the data by the other computing device; and (d) a wired or wireless medium for the transportation of data as a signal within a computing device from one element of the computing device to another element of the computing device for processing the data by the other element of the computing device. As may be used herein, a non-transitory computer readable memory is substantially equivalent to a computer readable memory. A non-transitory computer readable memory can also be referred to as a non-transitory computer readable storage medium.

(48) One or more functions associated with the methods and/or processes described herein can be implemented via a processing module that operates via the non-human “artificial” intelligence (AI) of a machine. Examples of such AI include machines that operate via anomaly detection techniques, decision trees, association rules, expert systems and other knowledge-based systems, computer vision models, artificial neural networks, convolutional neural networks, support vector machines (SVMs), Bayesian networks, genetic algorithms, feature learning, sparse dictionary learning, preference learning, deep learning and other machine learning techniques that are trained using training data via unsupervised, semi-supervised, supervised and/or reinforcement learning, and/or other AI. The human mind is not equipped to perform such AI techniques, not only due to the complexity of these techniques, but also due to the fact that artificial intelligence, by its very definition—requires “artificial” intelligence—i.e. machine/non-human intelligence.

(49) One or more functions associated with the methods and/or processes described herein can be implemented as a large-scale system that is operable to receive, transmit and/or process data on a large-scale. As used herein, a large-scale refers to a large number of data, such as one or more kilobytes, megabytes, gigabytes, terabytes or more of data that are received, transmitted and/or processed. Such receiving, transmitting and/or processing of data cannot practically be performed by the human mind on a large-scale within a reasonable period of time, such as within a second, a millisecond, microsecond, a real-time basis or other high speed required by the machines that generate the data, receive the data, convey the data, store the data and/or use the data.

(50) One or more functions associated with the methods and/or processes described herein can require data to be manipulated in different ways within overlapping time spans. The human mind is not equipped to perform such different data manipulations independently, contemporaneously, in parallel, and/or on a coordinated basis within a reasonable period of time, such as within a second, a millisecond, microsecond, a real-time basis or other high speed required by the machines that generate the data, receive the data, convey the data, store the data and/or use the data.

(51) One or more functions associated with the methods and/or processes described herein can be implemented in a system that is operable to electronically receive digital data via a wired or wireless communication network and/or to electronically transmit digital data via a wired or wireless communication network. Such receiving and transmitting cannot practically be performed by the human mind because the human mind is not equipped to electronically transmit or receive digital data, let alone to transmit and receive digital data via a wired or wireless communication network.

(52) One or more functions associated with the methods and/or processes described herein can be implemented in a system that is operable to electronically store digital data in a memory device. Such storage cannot practically be performed by the human mind because the human mind is not equipped to electronically store digital data.

(53) One or more functions associated with the methods and/or processes described herein may operate to cause an action by a processing module directly in response to a triggering event—without any intervening human interaction between the triggering event and the action. Any such actions may be identified as being performed “automatically”, “automatically based on” and/or “automatically in response to” such a triggering event. Furthermore, any such actions identified in such a fashion specifically preclude the operation of human activity with respect to these actions—even if the triggering event itself may be causally connected to a human activity of some kind.

(54) While particular combinations of various functions and features of the one or more embodiments have been expressly described herein, other combinations of these features and functions are likewise possible. The present disclosure is not limited by the particular examples disclosed herein and expressly incorporates these other combinations.

Claims

1. A method comprising: receiving an access request for a data segment, wherein the data segment is encoded utilizing an error correcting information dispersal algorithm as a set of encoded data slices that are stored in a plurality of storage units of a storage network and wherein each encoded data slice of the set of encoded data slices includes a corresponding checksum of a plurality of checksums; retrieving, from the storage network, a subset of encoded data slices that includes at least a threshold number of encoded data slices of the set of encoded data slices; determining, based on ones of the plurality of checksums corresponding to the subset of encoded data slices, when the subset of encoded data slices includes at least one corrupted encoded data slice; retrieving from at least one of the plurality of storage units an additional number of encoded data slices when required to generate a reconstructed data segment based on the subset of encoded data slices; generating the reconstructed data segment in accordance with the error correcting information dispersal algorithm, using the additional number of encoded data slices and at least some of the subset of encoded data slices; providing the reconstructed data segment in response to the access request; forming at least one reconstructed encoded data slice utilizing the error correcting information dispersal algorithm on the reconstructed data segment; and replacing the at least one corrupted encoded data slice with the at least one reconstructed encoded data slice.
2. The method of claim 1, wherein the threshold number of encoded data slices corresponds to a minimum number of the set of encoded data slices required to reconstruct the data segment.
3. The method of claim 1, wherein the error correcting information dispersal algorithm is a Cauchy-Reed-Solomon coding.
4. The method of claim 1, wherein the error correcting information dispersal algorithm is an erasure coding.
5. The method of claim 1, further comprising: determining, based on ones of the plurality of checksums corresponding to the subset of encoded data slices, when the subset of encoded data slices does not include a corrupted encoded data slice; decoding the threshold number of encoded data slices to recover the data segment; verifying accuracy of the recovered data segment; and when the accuracy of the recovered data segment has been verified, providing the recovered data segment in response to the access request.
6. The method of claim 1, wherein a first of the plurality of storage units is remotely located from a second of the plurality of storage units within the storage network.
7. The method of claim 1, wherein the plurality of checksums are based on a cyclic redundancy check.
8. A computer comprising: a port configured to support communications with a storage network; an application, coupled to the port, that is configured to enable a computer to perform operations that include: receiving an access request for a data segment, wherein the data segment is encoded utilizing an error correcting information dispersal algorithm as a set of encoded data slices that are stored in a plurality of storage units of a storage network and wherein each encoded data slice of the set of encoded data slices includes a corresponding checksum of a plurality of checksums; retrieving, from the storage network, a subset of encoded data slices that includes at least a threshold number of encoded data slices of the set of encoded data slices; determining, based on ones of the plurality of checksums corresponding to the subset of encoded data slices, when the subset of encoded data slices includes at least one corrupted encoded data slice; retrieving from at least one of the plurality of storage units an additional number of encoded data slices when required

to generate a reconstructed data segment based on the subset of encoded data slices; generating the reconstructed data segment in accordance with the error correcting information dispersal algorithm, using the additional number of encoded data slices and at least some of the subset of encoded data slices; providing the reconstructed data segment in response to the access request; forming at least one reconstructed encoded data slice utilizing the error correcting information dispersal algorithm on the reconstructed data segment; and replacing the at least one corrupted encoded data slice with the at least one reconstructed encoded data slice.

9. The computer of claim 8, wherein the error correcting information dispersal algorithm is a Reed-Solomon coding.

10. The computer of claim 8, wherein the error correcting information dispersal algorithm is a Cauchy-Reed-Solomon coding.

11. The computer of claim 8, wherein the error correcting information dispersal algorithm is an erasure coding.

12. The computer of claim 8, wherein the operations further comprise: determining, based on ones of the plurality of checksums corresponding to the subset of encoded data slices, when the subset of encoded data slices does not include a corrupted encoded data slice; decoding the threshold number of encoded data slices to recover the data segment; verifying accuracy of the recovered data segment; and when the accuracy of the recovered data segment has been verified, providing the recovered data segment in response to the access request.

13. The computer of claim 8, wherein a first of the plurality of storage units is remotely located from a second of the plurality of storage units within the storage network.

14. The computer of claim 8, wherein the plurality of checksums are based on a cyclic redundancy check.

15. A non-transitory computer readable storage medium comprises: at least one memory section that stores operational instructions that, when executed by a processing system of a storage network that includes a processor and a memory, causes the processing system to perform operations that include: receiving an access request for a data segment, wherein the data segment is encoded utilizing an error correcting information dispersal algorithm as a set of encoded data slices that are stored in a plurality of storage units of a storage network and wherein each encoded data slice of the set of encoded data slices includes a corresponding checksum of a plurality of checksums; retrieving, from the storage network, a subset of encoded data slices that includes at least a threshold number of encoded data slices of the set of encoded data slices; determining, based on ones of the plurality of checksums corresponding to the subset of encoded data slices, when the subset of encoded data slices includes at least one corrupted encoded data slice; retrieving from at least one of the plurality of storage units an additional number of encoded data slices when required to generate a reconstructed data segment based on the subset of encoded data slices; generating the reconstructed data segment in accordance with the error correcting information dispersal algorithm, using the additional number of encoded data slices and at least some of the subset of encoded data slices; providing the reconstructed data segment in response to the access request; forming at least one reconstructed encoded data slice utilizing the error correcting information dispersal algorithm on the reconstructed data segment; and replacing the at least one corrupted encoded data slice with the at least one reconstructed encoded data slice.

16. The non-transitory computer readable storage medium of claim 15, wherein the error correcting information dispersal algorithm is a Reed-Solomon coding.

17. The non-transitory computer readable storage medium of claim 15, wherein the error correcting information dispersal algorithm is a Cauchy-Reed-Solomon coding.

18. The non-transitory computer readable storage medium of claim 15, wherein the error correcting information dispersal algorithm is an erasure coding.

19. The non-transitory computer readable storage medium of claim 15, wherein the operations further comprise: determining, based on ones of the plurality of checksums corresponding to the

subset of encoded data slices, when the subset of encoded data slices does not include a corrupted encoded data slice; decoding the threshold number of encoded data slices to recover the data segment; verifying accuracy of the recovered data segment; and when the accuracy of the recovered data segment has been verified, providing the recovered data segment in response to the access request.

20. The non-transitory computer readable storage medium of claim 15, wherein a first of the plurality of storage units is remotely located from a second of the plurality of storage units within the storage network.
